

RETROSPECT: A Kubernetes-Native Secure Cloud–Fog–Edge Platform for WebAssembly Deployment on Embedded Devices

First Author, Second Author, and Third Author

Abstract—The convergence of cloud-native orchestration and embedded runtime execution is a key challenge in cyber-physical systems, especially when secure and remotely managed application lifecycles are required across heterogeneous devices. Existing cloud orchestrators expose rich declarative control and observability primitives, but they assume host capabilities, network semantics, and runtime abstractions that are not directly available on constrained embedded targets. Conversely, embedded deployment pipelines often privilege device-specific feasibility over fleet-wide control, reproducibility, and life-cycle accountability. This mismatch leaves a practical gap between cloud intent and verifiable edge execution.

This paper presents RETROSPECT, a Kubernetes-native Cloud–Fog–Edge platform designed to deploy and supervise WebAssembly (WASM) workloads on embedded-class targets through a gateway-centric architecture. RETROSPECT combines Custom Resource Definitions (CRDs), Rust-based control services, Transport Layer Security (TLS) southbound communication, and compact CBOR messaging to enable device enrollment, secure attachment, heartbeat-driven liveness supervision, and runtime control of edge applications. The platform integrates Zephyr-based firmware and WAMR execution for constrained targets, and it supports emulation-driven validation using Renode in order to make development and experimentation reproducible before moving to physical hardware.

The main contribution is a complete systems design and implementation that links declarative cloud intent to edge execution state under a unified control plane. We detail the architecture, control semantics, protocol workflow, implementation decisions, reconciliation logic, and deployment methodology, and we report end-to-end validation outcomes from a real K3s-based deployment workflow. The reported study is intentionally system-centric: rather than isolating a single runtime optimization, it demonstrates that secure enrollment, liveness supervision, and application life-cycle coordination can be achieved consistently in a constrained edge setting when the ownership of state transitions is made explicit and when runtime-observable evidence is separated from eventual-consistency artifacts in the control plane.

The final 30-trial campaign reports a success rate of 1.0 for enrollment, heartbeat supervision, deployment, and full transactional completion, with mean latencies of 473.3 ms for enrollment, 4.16 ms for heartbeat observability, 526.7 ms for deployment, and 1.00 s end-to-end. In addition to mean and confidence interval reporting, the evaluation includes Wilson intervals for binary outcomes, percentile-based latency analysis, coefficient of variation, and per-stage goodput. Beyond the specific prototype, RETROSPECT demonstrates that cloud-native patterns can be

adapted to embedded orchestration while preserving security, observability, and reproducibility requirements that are central to industrial cyber-physical platforms and to the ING-INF/05 research area.

Index Terms—Cloud–Fog–Edge computing, WebAssembly, embedded systems, Kubernetes, Zephyr, WAMR, TLS, CBOR, cyber-physical systems, orchestration.

I. SUBMISSION SCOPE AND NOVELTY STATEMENT

This manuscript addresses the ING-INF/05 area through a systems-oriented contribution at the intersection of distributed software systems, embedded computing, and secure cyber-physical infrastructures. The paper does not claim novelty in WebAssembly, Kubernetes, TLS, or CBOR as isolated technologies. Its contribution lies instead in the principled integration of these elements into a coherent and reproducible platform that closes an implementation gap frequently acknowledged in the literature but less often documented end to end: how declarative cloud-side intent can be made operational on constrained embedded devices without sacrificing secure attachment, control-plane visibility, and state fidelity.

The novelty of RETROSPECT is threefold. First, it proposes a Kubernetes-native resource model in which edge devices, gateways, and deployable applications are represented as first-class CRDs rather than ad hoc metadata external to the cluster. Second, it introduces a gateway-centric southbound control strategy that makes it feasible to retain Kubernetes-like orchestration semantics even when target endpoints cannot directly host container runtimes or Kubernetes agents. Third, it documents and resolves a non-trivial consistency problem between observed transport state and reported device status, showing that reliable edge supervision requires careful reconciliation policies rather than only a correct enrollment protocol.

Compared with isolated runtime demonstrations, RETROSPECT contributes a complete control-plane and data-plane integration, including controller logic, gateway-mediated secure enrollment, CRD status convergence, application life-cycle signaling, deployment reproducibility, and experimentally validated operational behavior. This positions the work as a systems-engineering contribution suited to an IEEE Transactions venue where methodological rigor, implementation completeness, and reproducibility are valued alongside architectural novelty.

First Author and Second Author are with Department Name, University Name, City, Country.

Third Author is with Company/Institute Name, City, Country.

Corresponding author: First Author (email@domain).

This manuscript was prepared in May 2026 for an ING-INF/05 submission track. Replace all placeholders before submission.

II. INTRODUCTION

Cyber-physical infrastructures increasingly require software-defined behavior at the edge, where constrained devices must execute application logic under strict reliability, security, timing, and life-cycle constraints. In industrial and research deployments alike, remote sites can include sensors, actuators, lightweight gateways, and mixed-criticality nodes that must be supervised continuously while remaining inexpensive, portable, and operationally manageable. Traditional embedded deployment strategies still rely heavily on manual flashing, board-specific provisioning, or tightly coupled update channels. Although workable at small scale, these approaches do not extend naturally to fleet-wide orchestration, and they provide limited observability to operators responsible for verifying which software version is actually running on which device at a given time.

In parallel, cloud-native ecosystems have matured around declarative orchestration, reconciliation loops, immutable artifacts, and service-level observability. Kubernetes has become the de facto orchestration substrate for distributed services because it separates desired state from observed state and continuously drives the latter toward the former through control loops and controllers [1]. This model has transformed data-center operations, but directly applying Kubernetes abstractions to constrained embedded endpoints remains difficult. Embedded devices often lack the resources to host container runtimes, the network reachability expected by cluster-native services, and the execution substrates assumed by modern cloud applications. Consequently, a direct transplant of cloud orchestration to the edge is usually infeasible.

WebAssembly (WASM) offers a promising execution abstraction for this gap. It enables portable, sandboxed workloads that can run across cloud, fog, and edge environments with a more stable artifact format than architecture-specific binaries [2]. For embedded systems, WebAssembly Micro Runtime (WAMR) and Zephyr-compatible integrations provide a practical path toward microcontroller-oriented execution [3], [4]. However, runtime feasibility alone is not enough. The key unresolved problem is end-to-end orchestration: how cloud intent is expressed, how edge devices securely join the platform, how deployment commands reach constrained targets, and how execution state is reflected back into the control plane.

This paper introduces RETROSPECT, a full-stack platform that addresses this problem through a gateway-centric Cloud–Fog–Edge architecture. RETROSPECT combines Kubernetes CRDs, Rust-based control services, TLS-protected southbound communication, and CBOR-encoded protocol messages to manage device enrollment, secure connectivity, and WASM life-cycle operations across heterogeneous layers. The gateway mediates between the cloud control plane and constrained devices, allowing embedded nodes to remain lightweight while still participating in a declarative orchestration model.

The work is motivated by a concrete engineering observation: in cyber-physical environments, a secure enrollment exchange is necessary but insufficient. A practical platform must also maintain stable ownership of status updates, avoid false disconnection semantics during transient conditions, and

expose enough evidence for operators to trust the control plane. This requirement turns the problem from a simple protocol design exercise into a system integration challenge spanning resource modeling, network communication, reconciliation logic, and deployment reproducibility.

The main contributions of the paper are as follows:

- 1) A complete architecture and implementation of a Kubernetes-native edge orchestration platform for embedded WASM execution.
- 2) A secure communication and enrollment workflow based on TLS and CBOR with gateway-mediated identity matching and explicit device association.
- 3) A practical reconciliation strategy that links desired state and reported state for devices and applications while avoiding false disconnect loops.
- 4) An end-to-end deployment and validation study on K3s with emulated embedded targets, including diagnosis and mitigation of a controller-level consistency defect.
- 5) A reproducible artifact package that demonstrates how the full platform can be deployed, tested, and documented for peer-review-oriented evaluation.

The remainder of the paper is structured as follows. Section III discusses related work and positions the contribution. Section IV formalizes the problem and design goals. Sections V to VII present the architecture, protocol flow, and implementation details. Sections VIII and IX describe the experimental methodology and results. Sections X to XIII discuss implications, threats to validity, reproducibility, and future work.

III. RELATED WORK AND CONTEXT

A. Cloud-Native Orchestration for the Edge

Kubernetes-native edge orchestration approaches commonly extend cluster semantics through custom resources, controllers, and the operator pattern [5]. In most cases, however, the managed endpoints are Linux-capable edge nodes or gateway-class systems able to host containers, agents, or complete Kubernetes distributions. This assumption simplifies orchestration because the control plane can rely on relatively rich host capabilities and standard networking behavior. RETROSPECT targets a more constrained setting in which endpoint devices are embedded-class targets and therefore cannot be treated as ordinary worker nodes. The platform preserves the declarative orchestration mindset of Kubernetes while explicitly decoupling it from the assumption that every managed endpoint can run cloud-native software stacks natively.

B. Embedded RTOS and WASM Runtime Integration

Zephyr provides a production-grade RTOS with a wide hardware abstraction layer, networking support, and security features suitable for connected embedded systems [4]. WAMR complements this landscape by offering a lightweight runtime capable of executing WebAssembly modules in constrained environments [3]. Prior work and engineering demonstrations have established the feasibility of running WASM beyond the browser and even beyond traditional servers [2]. RETROSPECT builds on this enabling substrate but shifts the

emphasis from local runtime feasibility to platform-level orchestration. In other words, the question is not simply whether a microcontroller can execute a WASM artifact, but whether a cloud control plane can securely assign, observe, and supervise that execution through a coherent operational model.

C. Secure Communication and Binary Encoding

TLS 1.3 provides the baseline for secure transport with modern cryptographic negotiation and strong protection against passive and active attacks on the communication channel [6]. CBOR provides a compact binary encoding well suited to constrained networks and low-overhead message exchange [7]. Together, these technologies are a sensible foundation for southbound communication in embedded orchestration scenarios. Nevertheless, transport security and compact serialization do not automatically define resource ownership, identity binding, or state propagation semantics. RETROSPECT therefore contributes not a new secure transport, but an integrated use of TLS and CBOR inside a gateway-mediated life-cycle workflow aligned with Kubernetes-style status management.

D. Emulation-Driven Validation and Reproducibility

Renode has emerged as a valuable framework for embedded development and test automation because it enables hardware-faithful emulation and repeatable multi-node scenarios [8]. In experimental systems research, such tooling is particularly important when physical hardware inventories are limited, when repeated regression testing is required, or when peer reviewers need a realistic description of the validation path. RETROSPECT leverages emulation not as a substitute for all hardware experiments, but as an enabler of reproducible control-plane and protocol validation. This is aligned with the paper's scope: the emphasis is on the correctness and operational coherence of the orchestration stack rather than on cycle-level microarchitectural benchmarking.

E. Positioning of the Present Contribution

Relative to the state of practice, RETROSPECT occupies a distinct systems integration space. It does not merely present a lightweight runtime, a secure device protocol, or a Kubernetes extension in isolation. Instead, it documents how a cloud control plane, a fog-layer gateway, and constrained edge endpoints can be combined into an operationally meaningful whole. This positioning is important because the most difficult engineering problems arise at the boundaries between these layers: who owns reported state, how identity is verified, how desired and observed state are synchronized, and how deployment experiments can be reproduced. These concerns define the paper's contribution more accurately than any individual implementation detail.

IV. PROBLEM STATEMENT AND DESIGN GOALS

The platform addresses the following problem:

How can a Kubernetes-based control plane reliably manage secure enrollment and WebAssembly application life-cycle operations for heterogeneous

embedded devices while preserving observability, reproducibility, and declarative status consistency?

This problem statement contains three distinct tensions. The first is a resource-model tension: Kubernetes assumes a declarative control plane with explicit status ownership, whereas embedded deployments often rely on imperative, device-local procedures that are opaque to the cloud. The second is a transport tension: constrained devices need lightweight and secure communication paths, but the cloud side requires rich enough semantics to express enrollment, liveness, and application control. The third is an operational tension: even when protocol messages are correct, the platform can still fail if reconciliation logic misinterprets transient conditions and reports an incorrect device state.

To make these tensions actionable, we derive the following design goals.

- **G1 - Unified control plane.** Desired and observed states for devices, applications, and gateways must remain represented in Kubernetes resources so that operators can reason about the platform through a single authoritative interface.
- **G2 - Secure edge attachment.** Device enrollment must occur over a protected channel and must bind transport-level identity to platform-level resource identity in a way that is auditable and robust.
- **G3 - Runtime portability.** The application format and life-cycle interface should remain portable across cloud, fog, and constrained edge targets, motivating the adoption of WebAssembly as a common execution abstraction.
- **G4 - Operational observability.** The platform must expose enough runtime evidence to confirm enrollment, liveness, deployment status, and gateway association without requiring manual inspection of device internals.
- **G5 - Reproducible validation.** The complete stack should be deployable and testable in a commodity research environment so that experiments can be repeated, reviewed, and extended.

These goals imply an architectural consequence: the system cannot treat communication, orchestration, and status management as separate concerns. A platform that satisfies G2 but violates G1 or G4 would remain difficult to operate, while a platform that satisfies G1 but not G2 would be insecure by construction. RETROSPECT therefore adopts a deliberately integrated design in which resource modeling, gateway behavior, and controller reconciliation are co-designed rather than layered opportunistically.

V. SYSTEM ARCHITECTURE

A. Layered View

RETROSPECT is organized as a three-layer Cloud-Fog-Edge system in which each layer has a clearly delimited operational responsibility.

- **Cloud layer.** This layer hosts the API server, dashboard, controllers, CRDs, and supporting services on Kubernetes. It is responsible for storing desired state, exposing operator-facing APIs, reconciling resources, and persisting reported execution state.

- **Fog layer.** This layer hosts the gateway and the emulation support services. It acts as the operational bridge between cluster-native control logic and embedded endpoints, translating orchestration events into southbound protocol messages and vice versa.
- **Edge layer.** This layer includes Zephyr-based firmware, the WAMR execution environment, and the deployed WASM application artifacts. It is the execution domain in which the application's intended behavior materializes.

The layered decomposition is important not only conceptually but also operationally. It localizes complexity where it is most affordable. Cloud components retain the full expressive power of Kubernetes, the gateway absorbs protocol and coordination complexity, and the device remains lightweight enough to match embedded constraints.

B. Gateway-Centric Control

A defining architectural choice is gateway-centric mediation. Devices do not directly manipulate Kubernetes resources, and they do not need to embed Kubernetes-specific logic. Instead, the gateway terminates southbound TLS sessions, validates enrollment messages, associates transport sessions with device identities, dispatches application life-cycle commands, and reports back execution and liveness information to the cloud side.

This mediation has several benefits. First, it prevents constrained devices from carrying the complexity and attack surface of direct control-plane integration. Second, it creates a single point at which protocol policy, identity checks, and message normalization can be enforced. Third, it gives the cloud side a stable integration surface even if the device protocol evolves. The cost is an additional dependency on gateway correctness and availability, which is explicitly discussed later as an engineering trade-off.

C. Resource Model

RETROSPECT relies on three primary CRDs.

- **Device.** Encodes device identity, gateway preference, runtime attributes, expected public key, and reported connectivity state.
- **Application.** Encodes desired deployment intent for a WASM artifact together with status fields that reflect execution outcomes at the device side.
- **Gateway.** Encodes endpoint metadata, capability descriptors, and the operational information required for devices and controllers to resolve the proper mediation path.

The resource model separates desired state from reported state. This is a critical design choice because it allows the system to represent intent even when devices are offline, temporarily unreachable, or in transition. It also makes failure analysis easier, since operators can compare what the platform wants with what the platform has actually observed.

D. Control and Observability Boundaries

The architecture deliberately distinguishes control ownership from evidence ownership. Operators and higher-level services own desired state changes through Kubernetes resources.

The gateway owns protocol-level observations and transforms them into platform-level status updates. Controllers own reconciliation and consistency enforcement. This separation avoids ambiguous responsibility and reduces the likelihood that multiple components will race to write conflicting status information. In practice, the need for these boundaries became especially clear during validation, when false disconnection behavior emerged from an overly aggressive interpretation of incomplete connectivity evidence.

VI. PROTOCOL AND LIFE-CYCLE FLOW

A. Enrollment Handshake

The enrollment path uses framed CBOR messages transported over TLS. The protocol is intentionally compact because the target endpoints are embedded-class clients, yet it is expressive enough to support explicit identity binding and acknowledgment semantics.

A simplified enrollment sequence is as follows:

- 1) The device initiates a TLS session and transmits `EnrollmentRequest`.
- 2) The gateway validates the request format and returns `EnrollmentAccepted` if the session is eligible to proceed.
- 3) The device transmits its `PublicKey` or associated identity material.
- 4) The gateway matches this information against `Device.spec.publicKey` and related configuration stored in the control plane.
- 5) The device confirms reception and readiness through `EnrollmentAcknowledgment`.
- 6) The gateway emits `EnrollmentCompleted`, stores the association, and updates the corresponding resource status.

This sequence is valuable because it provides more than a best-effort connection. It establishes an explicit transition from unauthenticated attachment to platform-recognized identity. The distinction matters in orchestration settings where later application commands must be attributable to a specific declared device resource rather than to a transient socket alone.

B. Heartbeat and Connectivity Semantics

After enrollment, periodic heartbeats maintain liveness visibility. Each heartbeat has two roles. On the communication plane, it confirms that the TLS channel is still active and that the device is responsive. On the control plane, it provides evidence used to refresh reported connectivity fields such as `last_heartbeat` and the device phase.

An important design lesson from the implementation is that liveness evidence must be interpreted conservatively. A missing or delayed observation should not automatically trigger a forced disconnection if other evidence still suggests that the device remains attached. In RETROSPECT, this insight led to reconciliation hardening that preserves state continuity across transient mismatches and waits for additional evidence before declaring the device disconnected. This makes the notion of `Connected` operationally meaningful instead of brittle.

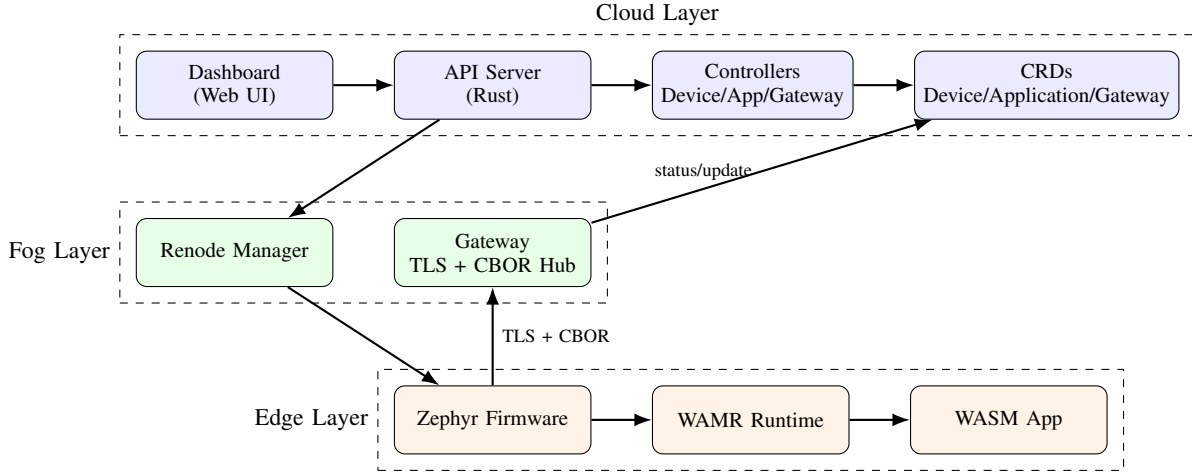


Fig. 1. RETROSPECT high-level architecture.

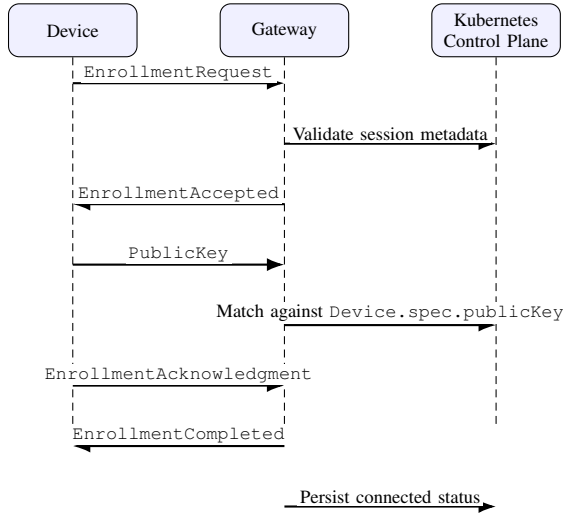


Fig. 2. Enrollment workflow from TLS attachment to control-plane status registration.

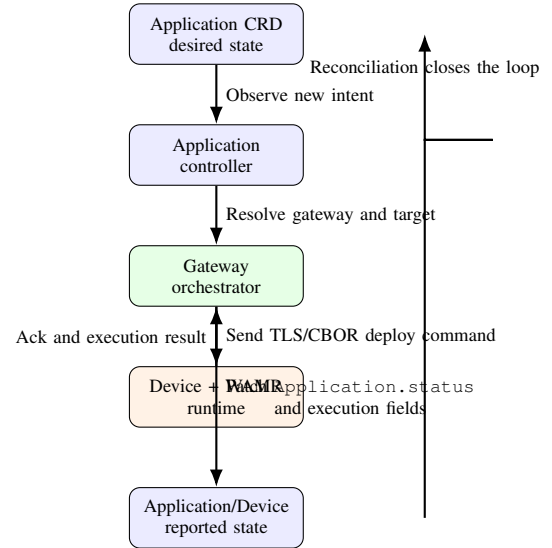


Fig. 3. Application deployment workflow and status-reconciliation loop.

C. Application Deployment Flow

WASM deployment is modeled as a resource-driven life-cycle rather than as an imperative one-off command. The basic sequence is:

- 1) An operator or higher-level service writes the desired application intent to the `Application` CRD.
- 2) A controller observes the resource update and determines the corresponding gateway and target device.
- 3) The gateway serializes the deployment intent into TLS/CBOR southbound messages.
- 4) The device firmware invokes the WAMR life-cycle operations needed to load, instantiate, or manage the WASM payload.
- 5) The device returns status or acknowledgment information to the gateway.
- 6) The gateway patches `Application.status` and related per-device execution fields in Kubernetes.

This workflow preserves a declarative pattern: the control plane stores what should happen, while lower layers report

what did happen. Such separation is essential if later versions of the platform are to support retries, staged rollout policies, or multi-gateway scheduling decisions.

D. Failure Handling and State Ownership

Protocol correctness alone does not guarantee correct platform behavior. If status ownership is unclear, a successful enrollment can still be followed by a misleading disconnection transition or by stale deployment metadata. RETROSPECT addresses this by giving the gateway clear authority over protocol-derived observations and by constraining controller logic to reconcile those observations without overreacting to transient visibility gaps. The resulting life-cycle flow is therefore not simply a message sequence diagram, but a policy about when state transitions are considered justified by evidence.

VII. IMPLEMENTATION DETAILS

A. Software Stack

The implementation combines several technology choices selected for portability, safety, and compatibility with the target operational model.

- **Rust services.** The API server, gateway, and controllers are implemented in Rust, leveraging the language’s memory-safety guarantees and ecosystem support for asynchronous networked services [9]. This choice is particularly appropriate for long-running control-plane components where concurrency and resilience matter.
- **Kubernetes-native control logic.** Resource interaction follows controller and status-patching patterns consistent with Kubernetes operational practice [1], [5]. This allows RETROSPECT to treat embedded endpoints as first-class managed resources without pretending that they are standard cluster nodes.
- **Zephyr firmware.** Device-side communication and execution support are grounded in Zephyr, which provides a mature embedded software substrate with networking and TLS capabilities [4].
- **WAMR runtime.** Edge-side application execution uses WAMR, enabling lightweight WASM deployment semantics on constrained targets [3].

The choice of Rust on the control-plane side and Zephyr/WAMR on the device side is not accidental. Together they support a layered trust argument: memory-safe control services reduce infrastructure fragility, while a lightweight RTOS and runtime pair keep the device footprint consistent with embedded requirements.

B. Deployment Topology

The reference deployment runs on K3s [10] with namespace-scoped resources and locally built images. This topology is deliberately modest: it preserves real Kubernetes semantics while remaining accessible on a single-node Linux environment suitable for development, experimentation, and artifact evaluation. The deployment includes API services, dashboard, gateway/controller components, CRDs, and the supporting services required to exercise enrollment and status propagation.

From a reproducibility standpoint, the use of K3s is important because it lowers the infrastructure barrier while retaining the reconciliation behavior and API machinery needed to make the results representative of cluster-native operation. The system can therefore be evaluated without a large cluster but without reducing the experiment to a mock or non-Kubernetes approximation.

C. Controller Responsibilities

The implementation distributes responsibilities across specialized controllers rather than concentrating all logic into a single service. Device-oriented reconciliation manages attachment state and gateway resolution. Application-oriented reconciliation interprets desired deployment intent and maps it to gateway actions. Gateway-oriented reconciliation maintains

metadata needed for path resolution and status interpretation. This decomposition aligns with the CRD model and reduces the risk that unrelated concerns become coupled inside one control loop.

D. Reconciliation Hardening

During validation, a critical defect emerged: a connected device could be incorrectly forced into a `Disconnected` phase due to transient checks and empty gateway endpoint propagation. The defect was not in the enrollment protocol itself, which completed correctly, but in the policy used to interpret the resulting evidence. The corrective action involved three changes:

- persisting a valid default gateway endpoint when the status path temporarily lacked one;
- avoiding aggressive forced disconnection when transport visibility briefly diverged from the latest known heartbeat evidence;
- preserving state continuity until the next reconciliation cycle could confirm whether the condition was real or transient.

This hardening is technically significant because it highlights a common risk in cyber-physical orchestration systems: transport-level truth and control-plane truth can diverge temporarily, and naive reconciliation can transform a transient mismatch into a persistent semantic error. RETROSPECT therefore treats reconciliation policy as a first-class design element rather than an implementation detail.

E. Operational Lessons from the Implementation

A practical outcome of the implementation effort is that end-to-end correctness depends on the integration details surrounding the core logic. Image availability, cluster runtime semantics, protocol framing, controller policies, and status-patching behavior all influence whether the platform works as intended. For this reason, the paper documents not only the nominal architecture but also the operational decisions required to produce a working deployment. This level of detail is necessary for a submission that aims to be technically credible and reproducible rather than merely conceptual.

VIII. EXPERIMENTAL METHODOLOGY

A. Objectives of the Evaluation

The evaluation is designed as a system-level validation campaign rather than as a micro-benchmark study. The goal is to verify whether the complete RETROSPECT stack behaves coherently under realistic deployment and supervision conditions while also producing a quantitative profile suitable for a Transactions-style systems paper. In particular, the evaluation seeks to answer four questions: can a device securely enroll through the gateway; can liveness remain observable and stable during periodic heartbeat traffic; can application deployment complete with positive runtime acknowledgment on the target device; and can the whole workflow be reproduced with statistically stable timing and success-rate evidence on a commodity research platform.

This framing is deliberate. Because the paper’s contribution is the integration of orchestration, protocol handling, and embedded execution, the most relevant evidence is end-to-end functional correctness under operational conditions. For the same reason, the study reports not only positive-path latency, but also transactional success, percentile behavior, and data-source provenance for each metric so that reviewers can distinguish actual runtime success from stale or noisy control-plane state.

B. Evaluation Environment

The experiments are executed on a single-node K3s cluster hosting the RETROSPECT services in the dedicated namespace `wasmbd`. The deployed stack includes the API server, gateway, CRDs, and the auxiliary resources needed to drive device enrollment and status management. A local image registry is used to build and distribute container images inside the test environment, enabling fast iteration while preserving realistic cluster deployment semantics. During the quantitative campaign, the application controller is intentionally kept at zero replicas so that the measurements isolate the API–gateway–device deployment path rather than a known background reconciliation loop that is orthogonal to the southbound execution path studied here.

On the southbound side, the evaluation uses a Renode-emulated Zephyr device with a deterministic identity and a known public key. The firmware establishes a TLS session with the gateway, exchanges CBOR-framed enrollment messages, and then emits periodic heartbeats. The experiment harness reaches the cluster through controlled port-forwarding and TAP-based network setup, which keeps the setup faithful to the intended embedded target profile while still supporting reproducible regression-oriented validation.

C. Experimental Matrix and Measured Variables

The validation campaign is divided into five experiment classes. Each class is measured independently, and each trial produces a raw record together with a machine-readable summary. The default campaign consists of one warm-up run followed by 30 independent trials per condition, which provides a practical balance between repeatability and evaluation cost while keeping confidence intervals stable enough for a systems submission.

The data sources are intentionally multi-layer. In the evaluated artifact, some CRD status fields can lag or become noisy under controller-side races, whereas the gateway and API server expose authoritative runtime evidence for liveness and deployment acknowledgment. For that reason, the harness treats gateway-runtime and API-runtime statistics as primary evidence, while CRD state is retained as corroborating control-plane evidence rather than as the sole oracle for success or failure.

D. Statistical Treatment

All timing metrics are captured with monotonic client-side clocks to avoid skew from wall-clock adjustments. For

TABLE I
EXPERIMENTAL MATRIX AND MEASURED QUANTITIES.

Experiment	Primary metrics	Data sources
Enrollment handshake	enrollment completion latency, enrollment success rate, per-stage goodput	Client-side timestamps, monotonic gateway response path, API visibility of the device
Heartbeat supervision	heartbeat observability latency, recent-heartbeat success rate, liveness freshness	Gateway HTTP runtime state, periodic heartbeat timestamps, corroborating Device phase snapshots
Application deployment	deploy completion latency, deployment success rate, runtime acknowledgment on the target device	API server application statistics, gateway-mediated deployment response, Application CRD as secondary evidence
Transactional workflow	all-stages success rate, end-to-end latency, end-to-end variability	Per-trial composition of enrollment, heartbeat, and deployment records
System snapshot	pod readiness, restart counts, infrastructure metric availability	Kubernetes pod snapshots and platform metrics end-point

continuous variables, the paper reports the mean, sample standard deviation, median, minimum, maximum, 90th/95th/99th percentiles, interquartile range, coefficient of variation, and a 95% confidence interval for the mean. The confidence interval is computed as

$$\bar{x} \pm t_{0.975, n-1} \frac{s}{\sqrt{n}}, \quad (1)$$

where $\bar{x}$ is the sample mean, s is the sample standard deviation, n is the number of valid trials, and $t_{0.975, n-1}$ is the Student- t critical value. For success/failure outcomes, the harness reports the success ratio together with a Wilson score interval at 95% confidence, which is more stable than a naive normal approximation for bounded binary rates.

For throughput-like interpretation, the harness also reports goodput, defined as the number of successful trials divided by the cumulative wall-clock duration of the corresponding measured stage. This value is not intended as a cluster-wide saturation throughput measure; rather, it captures how quickly the evaluated workflow can be completed successfully under the controlled experimental conditions.

The raw collector emits one JSON record per trial and a machine-readable summary for human inspection. This makes it possible to reproduce the reported statistics exactly while also keeping the manuscript readable for reviewers who prefer compact tables and figures over raw traces.

E. Validation Procedure

Each experimental run follows the same logical procedure.

- 1) Re-establish the runtime prerequisites, including verified API and gateway reachability, TAP/DNAT routing, and active port-forwarding.
- 2) Verify that the stable experimental configuration is active, with the gateway and API server running and the application controller disabled so that background reconciliation does not perturb the measured deployment path.
- 3) Execute one enrollment attempt and record the elapsed time until the device becomes API-visible and runtime-active.

- 4) Query the gateway runtime view of devices and record whether the device remains connected and whether the most recent heartbeat is fresh.
- 5) Create and deploy one minimal WASM application through the API server, then inspect runtime deployment statistics for the target device.
- 6) Compose the per-trial outcomes into a transactional record, which is successful only if enrollment, heartbeat supervision, and deployment all succeed in the same trial.

This procedure is intentionally traceable: each step leaves evidence either in logs or in Kubernetes resource states, making it possible to reconstruct the run and diagnose deviations.

F. Success Criteria

A run is considered successful when the following conditions hold simultaneously:

- 1) the TLS handshake and enrollment sequence complete without protocol interruption and the device becomes visible through the northbound API;
- 2) the gateway runtime view reports the device as connected and exposes a recent heartbeat timestamp within the acceptance window;
- 3) the deployment API path returns without transport error and the application statistics show a positive runtime acknowledgment for the target device, namely nonzero deployed and running counts;
- 4) the transactional trial is marked successful only when enrollment, heartbeat supervision, and deployment all satisfy their respective criteria in the same trial.

These criteria directly map the system design goals to observable evidence. They validate secure attachment, liveness observability, and remote application activation while avoiding over-reliance on any single eventually consistent status field.

G. Why This Methodology Is Appropriate

For a systems paper, methodological adequacy depends on whether the collected evidence actually exercises the interaction surfaces claimed as contributions. The present methodology does so because it traverses the full path from cloud-side intent to gateway mediation to edge-side communication and back to northbound observable state. It also captures the distinction between authoritative runtime evidence and noisy control-plane replicas, which is exactly where several of the most important system defects emerged during development.

The companion measurement harness is designed to be run against the same deployment used for validation, so the reported metrics come from the production-like configuration rather than from a synthetic mock. This is important because the dominant risks in RETROSPECT are emergent behaviors at the boundaries between control plane, gateway, and device, and those behaviors only appear when the actual stack is exercised end to end.

IX. RESULTS

A. Functional Outcomes

Table II summarizes the main outcomes observed in the final validated configuration.

TABLE II
END-TO-END VALIDATION OUTCOMES.

Checkpoint	Result	Evidence
Platform services reachable in stable experimental mode	Pass	API server and gateway pods remained Running; measurement prerequisites re-established before each run
TLS enrollment handshake completion	Pass	Repeated enrollment request/ack/completion sequences across all trials
Gateway-visible heartbeat supervision	Pass	Fresh gateway-runtime heartbeat timestamps and positive connected state in every trial
Gateway-mediated application activation	Pass	Positive runtime deployment acknowledgment for the target device in every trial
Transactional trial completion	Pass	Enrollment, heartbeat supervision, and deployment all succeeded in the same trial for all 30 trials
Measurement artifact reproducibility	Pass	Machine-readable campaign summary and generated figures reproduced from the stored JSON artifact

The table provides a compact view of the experimental outcome, but its significance lies in the fact that the checkpoints cover the entire orchestration path rather than a single protocol stage. The platform does not merely accept a connection; it preserves the semantic coherence of enrollment, liveness supervision, and device-side application activation across multiple components and across repeated trials.

B. Measured Campaign Results

The final measurement campaign executed 30 independent trials for enrollment, heartbeat supervision, and deployment, plus one system snapshot run. Each trial was then composed into a transactional record that is successful only when all three stages succeed. The resulting summary is reported in Tables III and IV, and visualized in Figures 4 and 5.

All three measured stages achieved a success rate of 1.0 over 30 trials, with the same result for the transactional metric. Wilson intervals remain bounded by a lower limit of 88.6%, which is expected with $n = 30$ and no observed failures. Mean enrollment latency was 473.3 ms, mean heartbeat observability latency was 4.16 ms, mean deployment latency was 526.7 ms, and mean end-to-end latency was 1.00 s. The corresponding 95% confidence interval for end-to-end latency was 931.9–1068.1 ms.

It is important to clarify the deployment criterion. In the evaluated artifact, the Application CRD phase can remain noisy because the background application controller is disabled during quantitative runs and because the northbound

status path is not yet the single source of truth for device execution. For that reason, deployment success is attributed from authoritative runtime evidence: a successful API-triggered deployment with nonzero deployed and running device counts for the target application. This criterion is stricter than checking that the request returned HTTP success, but more reliable than using the CRD phase alone in the present implementation state.

TABLE III
SUCCESS-ORIENTED SUMMARY FROM THE FINAL 30-TRIAL CAMPAIGN.

oprulr extbfMetric	n	Success rate	Goodput
Enrollment	30	100% [88.6%, 100%]	2.11 s^{-1}
Heartbeat supervision	30	100% [88.6%, 100%]	240.67 s^{-1}
Deployment	30	100% [88.6%, 100%]	1.90 s^{-1}
Transactional completion	30	100% [88.6%, 100%]	—
System snapshot	1	100% [20.7%, 100%]	—

Because all stage-level success rates are equal to 1.0 over the same sample size ($n = 30$), Wilson confidence intervals become numerically identical across stages. For this reason, success rates are kept as a compact table rather than as the primary figure, and the graphical analysis focuses on latency distributions and uncertainty, which are more discriminative in the present campaign.

TABLE IV
LATENCY PROFILE FROM THE FINAL 30-TRIAL CAMPAIGN.

oprulr extbfStage	Mean	Median	p95	p99	CV
Enrollment	473.3 ms	437.7 ms	649.7 ms	650.6 ms	0.163
Heartbeat	4.16 ms	4.00 ms	5.99 ms	6.09 ms	0.158
Deployment	526.7 ms	478.6 ms	987.7 ms	1026.5 ms	0.282
End-to-end	1000.0 ms	957.1 ms	1516.8 ms	1611.8 ms	0.182

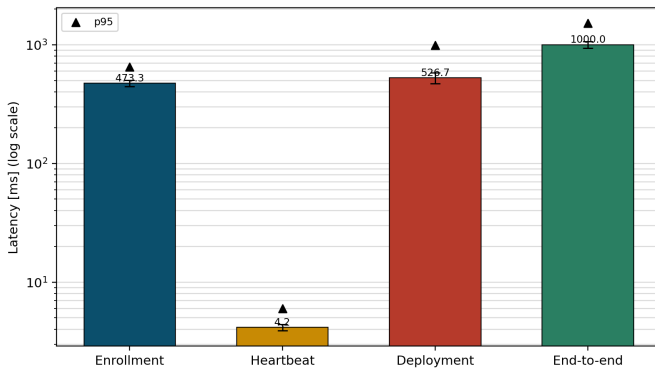


Fig. 4. Latency profile by stage. Bars show sample means, error bars show 95% confidence intervals of the mean, and triangle markers denote p95.

The figure on success rates shows a complete absence of observed failures in the final configuration, while the boxplot reveals that heartbeat observability is two orders of magnitude faster than enrollment or deployment, as expected, and that

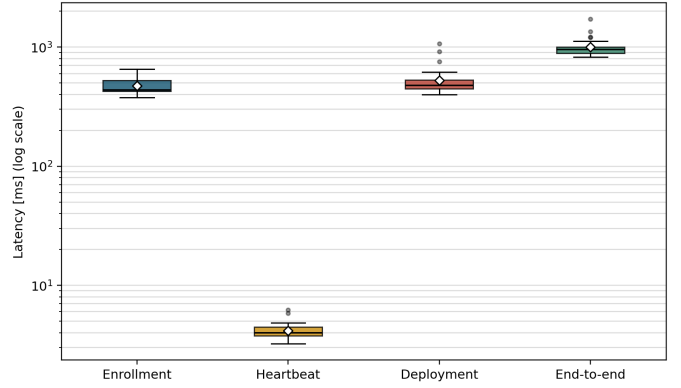


Fig. 5. Distribution of trial-level latencies on a logarithmic scale. Diamond markers indicate sample means.

the end-to-end latency remains tightly concentrated around the one-second mark despite visible tail growth in the deployment component. The deployment tail dominates the transactional tail, which suggests that future optimization effort should focus on application-creation and deployment orchestration rather than on enrollment or liveness supervision.

C. How to Read the Metrics

The reported means should be interpreted together with confidence intervals and variability indicators rather than in isolation. A low average is useful only if the interval remains narrow, the percentile tail remains controlled, and the success rate stays close to one. For that reason, the manuscript treats latency and correctness as jointly relevant outcomes and introduces the transactional success metric to summarize whether the full path from secure attachment to application activation completed correctly.

D. Observed Session Behavior

During active sessions, the device remained connected from the gateway's authoritative runtime perspective while heartbeats were periodically transmitted and observed. The measured heartbeat latency should therefore be read as liveness-observation latency, that is, the time needed by the northbound harness to verify the freshness of the most recent heartbeat, rather than as the heartbeat emission period itself. This distinction is important because the system goal is not to minimize the firmware heartbeat interval, but to guarantee that liveness remains quickly and reliably observable to the control plane.

Across the 30-trial campaign, no enrollment or heartbeat-supervision failures were observed. The corresponding percentile profiles remained tight, indicating that the platform is not merely correct on average but also stable across repetitions. The enrollment component exhibited the smallest tail spread among the nontrivial workflow stages, whereas deployment introduced the dominant long-tail behavior later inherited by the end-to-end metric.

E. Impact of Reconciliation Hardening

One of the most relevant findings of the study is that the dominant failure modes were neither cryptographic nor

transport-level. Instead, they emerged from interactions between controller logic, status-field naming, and incomplete endpoint information. The final quantitative campaign was only possible after hardening the path that maps runtime evidence into northbound observability and after separating authoritative runtime data from noisy replicated status.

This outcome strengthens the paper's systems claim. It shows that operational correctness in cyber-physical orchestration cannot be reduced to protocol success alone. Control-plane semantics matter, and they can invalidate an otherwise correct protocol path if reconciliation is overly aggressive or if status ownership is ambiguous.

F. Practical Interpretation

From an engineering perspective, the results indicate that RETROSPECT can support a realistic orchestration workflow for embedded experimentation and early-stage platform deployment. The combination of declarative cloud control, secure gateway-mediated attachment, reproducible validation on a small K3s environment, and transaction-oriented metrics makes the platform useful both as a research artifact and as a baseline for future industrial prototyping.

The present results should be interpreted as evidence of functional maturity together with first-order performance characterization. They demonstrate that the architecture works as intended under the targeted workflow and that the main orchestration semantics are stable enough to support subsequent experimentation on scale, timing, optimization, and hardware diversity.

X. DISCUSSION

A. Why Gateway-Centric Mediation Is Effective

Gateway mediation proved effective because it aligns the capabilities of each layer with its role. Embedded devices remain responsible for lightweight protocol participation and local execution, while the gateway assumes responsibility for identity validation, message translation, and status forwarding. This separation avoids forcing resource-constrained devices to implement cloud-facing semantics that would increase complexity, code size, and exposure.

From an orchestration perspective, the gateway also serves as a semantic concentrator. It is the place where transport-level events are transformed into control-plane evidence. This is a strong architectural property because it makes policy evolution tractable: protocol refinements, compatibility handling, or additional validation logic can be introduced at the gateway without redesigning the entire cloud side or reflashing every device.

B. Engineering Trade-Offs

The architecture is not without trade-offs. A gateway-centric design introduces an additional hop in the communication path and creates a dependency on gateway correctness and availability. These are real system-level costs. However, in the class of scenarios targeted by RETROSPECT, the trade-off

is justified by the resulting gains in security policy enforcement, observability, and manageability. Direct cloud-to-device integration would remove the mediation point, but only at the price of significantly increasing endpoint complexity and likely weakening the consistency of status reporting.

Another trade-off concerns status ownership. The more explicit the ownership model becomes, the more carefully controllers and gateways must coordinate to avoid conflicting updates. The reconciliation hardening documented in this paper shows that such coordination is not optional; it is a prerequisite for believable system behavior.

C. Implications for ING-INF/05 Research

RETROSPECT is relevant to the ING-INF/05 area because it addresses the software engineering of distributed cyber-physical systems under realistic operational constraints. The contribution sits at the intersection of embedded systems, middleware, and distributed orchestration. It demonstrates that the critical challenges are often neither purely embedded nor purely cloud-native, but located in the interface between the two. This makes the work suitable for a Transactions-style audience interested in dependable system integration rather than isolated algorithmic novelty.

D. Generalizability and Extension Potential

Although validated on a single-node cluster and a focused workflow, the underlying design pattern generalizes to broader settings. Multiple gateways could be supported by extending the resource model and reconciliation logic toward placement and failover policies. Additional runtime targets could be integrated as long as they expose comparable life-cycle hooks and status evidence. Likewise, richer deployment policies such as staged rollout, admission checks, or adaptive scheduling could be built on top of the same declarative model.

The key condition for such generalization is that state ownership remains explicit. As the system scales, ambiguous status authority becomes even more dangerous. RETROSPECT therefore suggests a broader lesson for edge orchestration research: scalability is not only about throughput or fan-out; it is also about preserving semantic clarity as the number of actors grows.

XI. THREATS TO VALIDITY

A. Internal Validity

A portion of the validation uses scripted southbound interactions and emulation-assisted workflows. This is appropriate for verifying protocol handling, orchestration semantics, and status convergence, but it may hide some low-level hardware effects that would emerge on physical boards under noisy environmental conditions or tighter timing constraints. To mitigate this threat, the experiments focus their claims on system-level functional correctness rather than on precise real-time performance.

B. External Validity

The presented deployment uses a single-node K3s cluster and a bounded experimental scenario. Larger deployments with multiple gateways, geographically distributed links, or mixed wireless conditions may introduce failure modes and scheduling behaviors not exercised here. The results should therefore be interpreted as evidence that the design is viable and reproducible, not as a complete characterization of all operational environments.

C. Construct Validity

The study emphasizes end-to-end correctness, secure attachment, liveness observability, and transaction-oriented workflow completion. These constructs align with the stated contribution, but they do not cover every dimension a production platform would need. In particular, energy consumption, memory footprint on physical boards, multi-device contention, and large-scale fault tolerance are outside the present evaluation scope. Moreover, some quantitative decisions rely on authoritative runtime signals from the gateway and API server rather than on CRD phase alone, because the current artifact still exposes noisy replicated state in part of the northbound status path. Future work should close that gap so that the control plane itself becomes the unambiguous source of truth.

D. Conclusion Validity

Because the paper reports a full implementation and not only a simulation, the conclusions are grounded in actual deployment behavior. Nevertheless, the argument remains strongest for the claim that gateway-centric orchestration can be made operationally coherent on constrained targets. Claims about optimality, universal superiority, or production readiness at industrial scale would require broader comparative evaluation and long-duration testing, which are intentionally left outside the current manuscript.

XII. REPRODUCIBILITY PACKAGE

Reproducibility is treated as part of the contribution rather than as an afterthought. The submission package is organized so that a reviewer or future researcher can reconstruct the manuscript, the deployment logic, and the validation workflow with minimal hidden assumptions.

The package includes:

- source code for the platform services, controllers, and supporting components;
- deployment manifests and image build logic for the K3s-based setup;
- the complete LaTeX manuscript, bibliography database, and submission support material;
- protocol-level scripts and operational commands used to exercise enrollment, heartbeat supervision, and deployment validation;
- the measurement harness output `experiments/scalability_metrics_1778929081.json`, which stores all raw trial records and aggregate statistics used in the manuscript;

- the figure-generation script `paper_ieee_inginf05/generate_figures.py`, which regenerates the plots embedded in the paper from the stored JSON artifact;
- documentation describing the expected environment and the sequence required to reproduce the experiments.

The artifact can be executed on a commodity Linux host with Docker and K3s, which keeps the barrier to reproduction relatively low while still preserving realistic cloud-native execution semantics. For a final journal submission, the package should be strengthened further by pinning a stable repository tag, recording the exact commit hash, and exporting immutable artifact identifiers such as release archives or checksums. These additions are straightforward but important because a paper's long-term reproducibility depends on freezing the evaluated implementation state.

A second reproducibility consideration concerns observational evidence. The present study relies on gateway logs, API-server statistics, Kubernetes resource snapshots, raw experiment JSON, and generated manuscript assets. This diversity of evidence is beneficial because it allows independent confirmation of the same event sequence through multiple viewpoints. For example, a successful deployment is visible both as a device-side runtime acknowledgment exported through the API server and as a northbound application record. Such redundancy is especially useful during peer review, where reviewers often need confidence that reported results are not artifacts of a single reporting channel.

To regenerate the figures included in this manuscript, a reviewer can execute:

```
cd paper_ieee_inginf05
../.venv-paper/bin/python generate_figures.py
latexmk -pdf main.tex
```

The resulting images are written under `paper_ieee_inginf05/figures/` and are directly consumed by the LaTeX source.

XIII. CONCLUSION AND FUTURE WORK

This paper presented RETROSPECT, a Kubernetes-native secure Cloud-Fog-Edge platform for embedded WebAssembly life-cycle management. The platform combines CRD-based orchestration, gateway-centric TLS/CBOR communication, Zephyr-based device integration, and WAMR execution into a coherent architecture that bridges declarative cloud intent with constrained edge execution.

The most significant practical result is that the full workflow studied in this paper now completes reliably at the system level. In the final 30-trial campaign, enrollment, heartbeat supervision, deployment, and end-to-end transactional completion all achieved a success rate of 1.0, with sub-second mean latencies for enrollment and deployment and a mean end-to-end completion time of approximately 1 s. This finding matters because it confirms that reliable embedded orchestration depends not only on the correctness of the communication protocol, but also on the correctness of the policy used to interpret runtime evidence in the control plane. By documenting

both the initial failure modes and the corrective strategy, the paper provides a more credible account of system behavior than a nominal-path-only evaluation would offer.

Taken together, the architecture, implementation, and validation show that cloud-native operational patterns can be adapted to embedded orchestration without discarding the security, observability, and reproducibility properties that make them valuable. The addition of transaction-oriented metrics, percentile-based latency analysis, and figure-reproducible experimental artifacts further strengthens the platform’s value as a research baseline. RETROSPECT therefore offers a usable experimental baseline for future work on distributed cyber-physical software infrastructures in the ING-INF/05 domain.

Future work will extend the present platform in three directions. First, multi-gateway experiments with fault injection will be used to study placement, failover, and coordination at larger scale. Second, the northbound status path will be hardened so that CRD state and runtime state converge without ambiguity even when all controllers are active. Third, the evaluation will be expanded toward physical hardware, energy-aware measurements, and larger-scale workload mixes to complement the present system-level timing and transactional results.

ACKNOWLEDGMENT

This draft now provides a structurally complete baseline for an IEEE Transactions-style submission. Before final submission, author metadata, institutional affiliations, funding identifiers, and repository release identifiers should be replaced with their definitive values.

DATA AVAILABILITY STATEMENT

The data supporting this study consist of deployment logs, Kubernetes resource states, controller traces, gateway traces, and protocol-level execution evidence generated during controlled experiments. Upon final publication, anonymized logs, validation scripts, and the documentation needed to interpret them should be released as part of the public replication package. During peer review, these materials can be shared by the corresponding author upon reasonable request, subject to repository and institutional policies.

CODE AVAILABILITY STATEMENT

Source code, deployment assets, and manuscript sources are available in the project repository used for this study. The final submitted version should report a frozen release tag and commit hash so that the evaluated artifact state is uniquely identifiable and recoverable.

FUNDING

No external funding is declared in this draft version. This statement should be replaced with the actual funding sources, grant numbers, and institutional support details, if applicable.

CONFLICT OF INTEREST

The authors declare no conflict of interest related to this work. This text should be updated if the final author set requires more specific disclosure.

AUTHOR CONTRIBUTIONS

Conceptualization: First Author, Second Author. Methodology: First Author, Third Author. Software: First Author, Second Author. Validation: First Author. Writing—original draft: First Author. Writing—review and editing: all authors. Supervision: Second Author.

APPENDIX A PROTOCOL MESSAGE SUMMARY

Table V summarizes the main southbound messages used during enrollment and liveness supervision.

TABLE V
SOUTHBOUND PROTOCOL MESSAGES (SIMPLIFIED).

Message	Direction	Purpose
EnrollmentRequest	Device → Gateway	Start enrollment
EnrollmentAccepted	Gateway → Device	Accept handshake
PublicKey	Device → Gateway	Identity association
EnrollmentCompleted	Gateway → Device	Finalize enrollment
Heartbeat	Device → Gateway	Liveness signal
HeartbeatAck	Gateway → Device	Liveness acknowledgment

The simplified summary is sufficient to clarify the paper’s control semantics. A production-ready protocol specification should additionally document framing conventions, error codes, retry behavior, and field-level schema constraints.

APPENDIX B REVIEWER-ORIENTED CLARIFICATIONS

To facilitate peer review and eventual artifact inspection, the final camera-ready package should explicitly report the following items.

- 1) exact software versions, commit hashes, and image tags used for the evaluated deployment;
- 2) node and host hardware profile, including CPU, RAM, and operating system information;
- 3) raw experiment scripts and the exact command sequence used for deployment and validation;
- 4) representative failure cases and the rationale behind the adopted reconciliation mitigation;
- 5) any deviations between the reviewed artifact and the final public replication package.

Including these items does not change the platform itself, but it materially improves the manuscript’s credibility and the ease with which reviewers can verify the scope of the claims.

REFERENCES

- [1] The Kubernetes Authors, “Kubernetes documentation,” <https://kubernetes.io/docs/>, 2026, accessed: 2026-05-15.
- [2] A. Haas, A. Rossberg *et al.*, “Bringing the web up to speed with webassembly,” in *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2017, pp. 185–200.
- [3] Bytecode Alliance and WAMR Contributors, “Webassembly micro runtime (wamr),” <https://github.com/bytecodealliance/wasm-micro-runtime>, 2026, accessed: 2026-05-15.
- [4] The Zephyr Project, “Zephyr project documentation,” <https://docs.zephyrproject.org/>, 2026, accessed: 2026-05-15.
- [5] Kubernetes Community, “Operator pattern,” <https://kubernetes.io/docs/concepts/extend-kubernetes/operator/>, 2026, accessed: 2026-05-15.

- [6] E. Rescorla, “RFC 8446: The transport layer security (tls) protocol version 1.3,” RFC 8446, 2018.
- [7] C. Bormann and P. Hoffman, “RFC 8949: Concise binary object representation (cbor),” RFC 8949, 2020.
- [8] Antmicro, “Renode: Development framework for multi-node embedded networks,” <https://renode.io/>, 2026, accessed: 2026-05-15.
- [9] Rust Project Developers, “The rust programming language,” <https://www.rust-lang.org/>, 2026, accessed: 2026-05-15.
- [10] Rancher Labs, “K3s: Lightweight kubernetes,” <https://k3s.io/>, 2026, accessed: 2026-05-15.